

Unidad 0 Sistemas de numeración

José Luis Martínez García

Índice

1. Representación de la información	1
2. Codificación de la información	13

1. Representación de la información

Resultado de aprendizaje

Unidad **introductoria** del **1er trimestre**. No tiene criterios de evaluación propios: proporciona la base (sistemas de numeración, operaciones binarias y codificación de la información) necesaria para el **RA1**: *Evalúa sistemas informáticos identificando sus componentes y características*, en especial para:

- a) Se han reconocido los componentes físicos de un sistema informático y sus mecanismos de interconexión.
- d) Se han identificado los tipos de redes y sistemas de comunicación.

1. Representación de la información

La función principal del **ordenador** es automatizar la información. Tanto para almacenar la información que le suministremos, como para procesarla o transmitirla a otro ordenador, es **necesario que la transforme a un sistema que pueda entender**.

Cualquier dato que le suministremos al ordenador será transformado internamente en un conjunto de 0 y 1. A cada 0 o 1 se le llama *dígito binario* o *bit* (lo veremos más adelante en 2.1 Almacenamiento de la información).

De forma contraria, cuando necesitamos un dato almacenado en el ordenador, este transformará el conjunto de 0 y 1 en una entidad legible por el usuario.

1.1 Tipos de datos según el lugar que ocupan en el proceso

- **Datos de entrada:** los que introduce el usuario al ordenador, ya sea mediante el teclado, un micrófono o cualquier otro periférico de entrada.
- **Datos internos del proceso:** los que utiliza el ordenador internamente para realizar operaciones antes de mostrar los resultados.
- **Datos de salida:** los resultados del proceso de automatizar la información.



1.2 Sistemas de numeración

Un **sistema de numeración** es un conjunto de símbolos y reglas que se utilizan para representar cantidades o valores numéricos, p. ej. $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

Se clasifican en:

- **Posicionales:** el valor de cada símbolo depende de su valor propio y de la posición que ocupa. P. ej. el sistema decimal.
Se caracterizan por la **base**, que indica el número de símbolos diferentes que se pueden usar en ese sistema para representar cualquier cantidad.
- **No posicionales:** cada símbolo tiene un valor independiente de la posición que ocupe. P. ej. los números romanos.

Teorema fundamental de la numeración (TFN) Relaciona una cantidad expresada en cualquier sistema de numeración con la misma cantidad expresada en decimal:

$$N = \sum_{i=-j}^{k-1} X_i \cdot B^i$$

- N : valor de la cantidad expresada en el sistema decimal.
- k : número de dígitos de la parte entera (a la izquierda de la coma).
- j : número de dígitos de la parte fraccionaria (a la derecha de la coma).
- B : base del sistema de numeración en que está expresado el número.
- X_i : cada uno de los dígitos del número (el dígito inmediatamente a la izquierda de la coma ocupa la posición 0).

Actividad 1

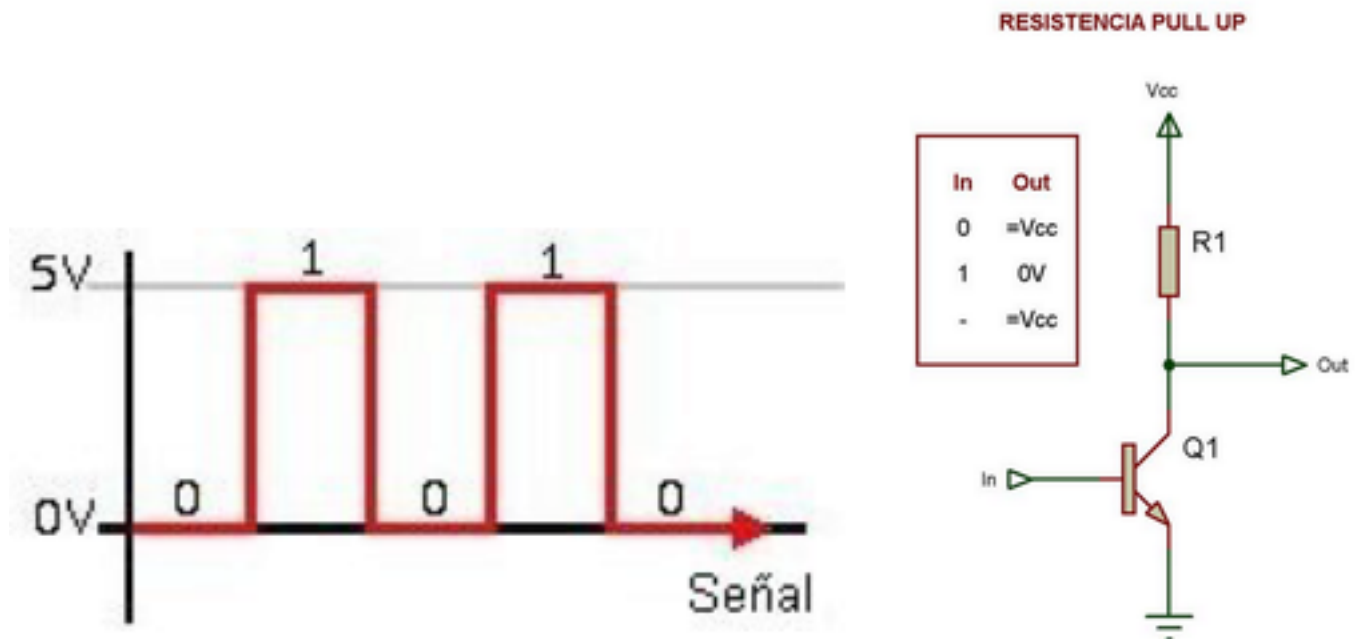
Utiliza el TFN para ver cómo se calcularía el valor de cada dígito de los siguientes números decimales:

1. 1785
2. 345,65
3. 12,357

Sistemas usados en informática Además del sistema decimal (base 10), en informática se usan mucho:

Binario (base 2) Utiliza dos símbolos: $\{0, 1\}$.

Es el sistema que utiliza internamente el ordenador. En realidad el ordenador no trabaja con 0 y 1, sino con señales eléctricas: un voltaje positivo (p. ej. 5 V) puede significar un 1 y un voltaje nulo (0 V) un 0.



Octal (base 8) Utiliza los símbolos $\{0, 1, 2, 3, 4, 5, 6, 7\}$. Tiene una correlación directa con el binario: **cada símbolo octal se representa con 3 bits.**

Decimal	Octal	Binario
0	0	000
1	1	001
2	2	010
3	3	011
4	4	100
5	5	101
6	6	110
7	7	111

Hexadecimal (base 16) Utiliza 16 símbolos: los números del 0 al 9 y las seis primeras letras del abecedario, es decir $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$. También tiene correlación directa con el binario: **cada símbolo hexadecimal se representa con 4 bits.**

Decimal	Hexadecimal	Binario	Decimal	Hexadecimal	Binario
0	0	0000	8	8	1000
1	1	0001	9	9	1001
2	2	0010	10	A	1010
3	3	0011	11	B	1011
4	4	0100	12	C	1100
5	5	0101	13	D	1101
6	6	0110	14	E	1110
7	7	0111	15	F	1111

1.3 Cambios de base

Binario → decimal Se aplica el TFN: se multiplica cada dígito por la potencia de 2 correspondiente a su posición y se suma todo.

- $11011_2 = 1 \cdot 2^0 + 1 \cdot 2^1 + 0 \cdot 2^2 + 1 \cdot 2^3 + 1 \cdot 2^4 = 1 + 2 + 8 + 16 = 27_{10}$
- $11,011_2 = 1 \cdot 2^{-3} + 1 \cdot 2^{-2} + 0 \cdot 2^{-1} + 1 \cdot 2^0 + 1 \cdot 2^1 = 0,125 + 0,25 + 0 + 1 + 2 = 3,375_{10}$
- $11101_2 = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 1 \cdot 2^3 + 1 \cdot 2^4 = 1 + 0 + 4 + 8 + 16 = 29_{10}$

Decimal → binario

- **Parte entera:** se divide sucesivamente entre 2 hasta que el **cociente** sea menor que 2. El número se forma con el último cociente seguido de todos los restos en orden inverso.
- **Parte fraccionaria:** se multiplica por 2; la parte entera obtenida (0 o 1) es el siguiente bit y se descarta para la siguiente multiplicación. Se repite hasta que la parte fraccionaria sea 0 (o hasta tener la precisión deseada).

Ejemplo: $32\{,\}375_{10}$

Parte entera: $32 : 2 = 16 \text{ r } 0 \rightarrow 16 : 2 = 8 \text{ r } 0 \rightarrow 8 : 2 = 4 \text{ r } 0 \rightarrow 4 : 2 = 2 \text{ r } 0 \rightarrow 2 : 2 = 1 \text{ r } 0 \rightarrow 32_{10} = 100000_2$

Parte fraccionaria:

- $0,375 \cdot 2 = 0,75 \rightarrow \text{bit } 0$
 - $0,75 \cdot 2 = 1,5 \rightarrow \text{bit } 1$ (se elimina la parte entera)
 - $0,5 \cdot 2 = 1,0 \rightarrow \text{bit } 1$
- $0,375_{10} = 0,011_2 \rightarrow 32,375_{10} = 100000,011_2$

Binario ↔ octal Cada dígito octal equivale a un grupo de **3 bits**.

Octal → binario

- $1342_8 = 001\ 011\ 100\ 010_2$
- $34,03_8 = 011\ 100,000\ 011_2$

Binario → octal

- $11011101_2 = 011\ 011\ 101 = 335_8$
- $10001,1001_2 = 010\ 001,100\ 100 = 21,44_8$

Cómo agrupar

Se toman los bits en grupos de 3 **desde la coma hacia la izquierda** para la parte entera y **desde la coma hacia la derecha** para la fraccionaria. Si faltan bits para completar un grupo, se añaden ceros donde no alteren el valor: a la izquierda en la parte entera y a la derecha en la fraccionaria.

Binario ↔ hexadecimal Cada dígito hexadecimal equivale a un grupo de **4 bits** (mismas reglas de agrupación, con grupos de 4).

Hexadecimal → binario

- $3AF_{16} = 0011\ 1010\ 1111_2$
- $F0,0C_{16} = 1111\ 0000,0000\ 1100_2$

Binario → hexadecimal

- $110110010_2 = 0001\ 1011\ 0010 = 1B2_{16}$
- $101011,010001_2 = 0010\ 1011,0100\ 0100 = 2B,44_{16}$

Octal ↔ decimal **Octal → decimal (TFN):** se multiplica cada dígito por la potencia de 8 correspondiente y se suma.

- $35_8 = 3 \cdot 8^1 + 5 \cdot 8^0 = 29_{10}$

Decimal → octal: divisiones sucesivas entre 8 hasta que el cociente sea menor que 8; se toma el último cociente y los restos en orden inverso.

- $29 : 8 = 3 \text{ resto } 5 \rightarrow 29_{10} = 35_8$ (el resto 5 es el dígito menos significativo, LSD; el cociente 3 el más significativo, MSD)

Hexadecimal → decimal Se aplica el TFN con potencias de 16, sustituyendo cada letra por su valor decimal.

$$F12A4_{16} = 15 \cdot 16^4 + 1 \cdot 16^3 + 2 \cdot 16^2 + 10 \cdot 16^1 + 4 \cdot 16^0$$

$$= 983040 + 4096 + 512 + 160 + 4 = 987812_{10}$$

Decimal → hexadecimal Se divide sucesivamente entre 16 hasta que el cociente sea menor que 16. Después se sustituye cada resto y el último cociente por su símbolo hexadecimal y se colocan de derecha a izquierda (el primer resto es el dígito menos significativo).

- **Ejemplo 1:** $29 : 16 = 1 \text{ resto } 13 (D) \rightarrow 29_{10} = 1D_{16}$
- **Ejemplo 2:** $1972 : 16 = 123 \text{ resto } 4; 123 : 16 = 7 \text{ resto } 11 (B) \rightarrow 1972_{10} = 7B4_{16}$

1.4 Operaciones en binario

En binario se pueden realizar:

- **Operaciones aritméticas:** las mismas que en decimal (suma, resta, multiplicación, división...).
- **Operaciones lógicas:** con operadores como NOT, AND, OR... → ver 1.4.2 Operaciones lógicas.

1.4.1 Operaciones aritméticas

Suma

A	B	Z (salida)	C (acarreo)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Ejemplo 1 ($79 + 23 = 102$)

```

      11 111      -> C (acarreo)
    0100 1111
+   0001 0111
-----
    0110 0110      -> Z (salida)
```

Ejemplo 2 ($108 + 45 = 153$)

```

      11 1 1      -> C (acarreo)
    0110 1100
+   0010 1101
-----
    1001 1001      -> Z (salida)
```

Resta En la resta, el “acarreo” es en realidad un **préstamo** (*borrow*) que se resta a la columna siguiente.

A	B	Z (salida)	C (acarreo/préstamo)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Ejemplo 1 ($111 - 23 = 88$)

```

  0110 1111
    1      -> C (préstamo)
- 0001 0111
-----
  0101 1000      -> Z (salida)

```

Ejemplo 2 ($108 - 29 = 79$)

```

  0110 1100
    11 111      -> C (préstamo)
- 0001 1101
-----
  0100 1111      -> Z (salida)

```

Consejo

En la práctica, el ordenador no resta así: suma el complemento del sustraendo. Ver 1.4.3 Complementos.

1.4.2 Operaciones lógicas

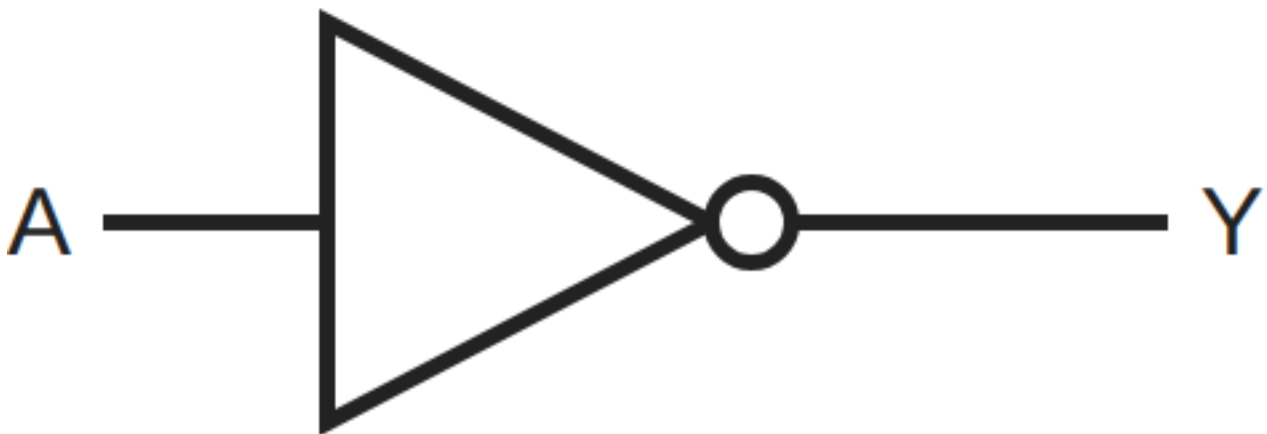
Simbología

Cada puerta tiene un símbolo propio. El **círculo** a la salida indica **negación** (NOT, NAND, NOR).

Se basan en el **álgebra de Boole**, llamada así por el matemático inglés George Boole, que sentó las bases de las operaciones lógicas binarias. Los circuitos internos del ordenador contienen **puertas lógicas**: todas las operaciones aritméticas se realizan, en realidad, mediante operaciones lógicas.

NOT El resultado es la negación de la entrada.

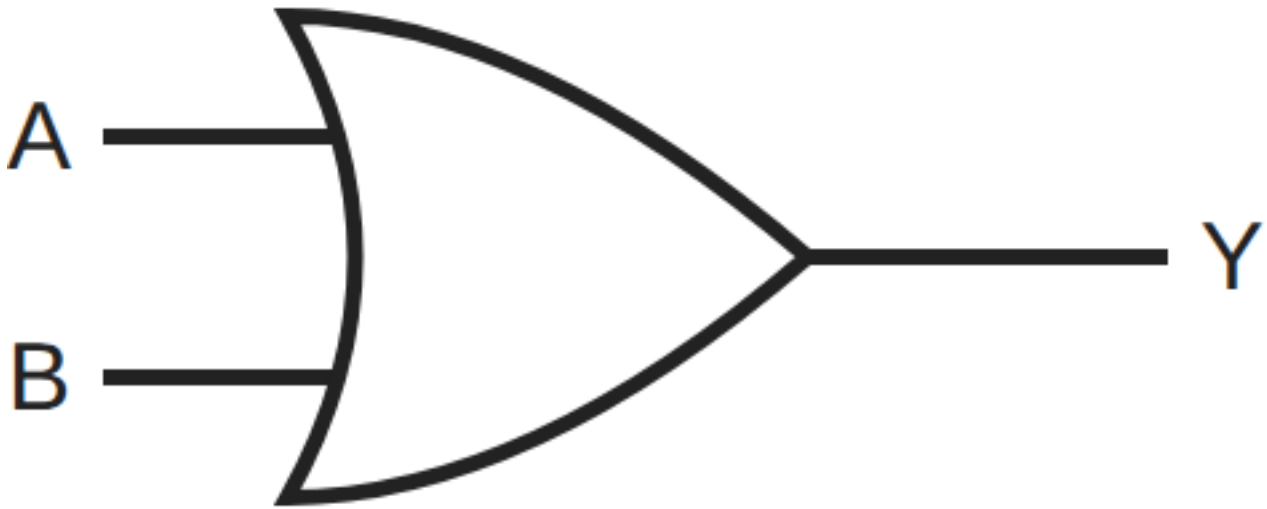
Símbolo (izquierda: ANSI/IEEE) · **Expresión:** $Y = \overline{A}$



A	Y
0	1
1	0

OR El resultado es 0 solo cuando ambas entradas son 0.

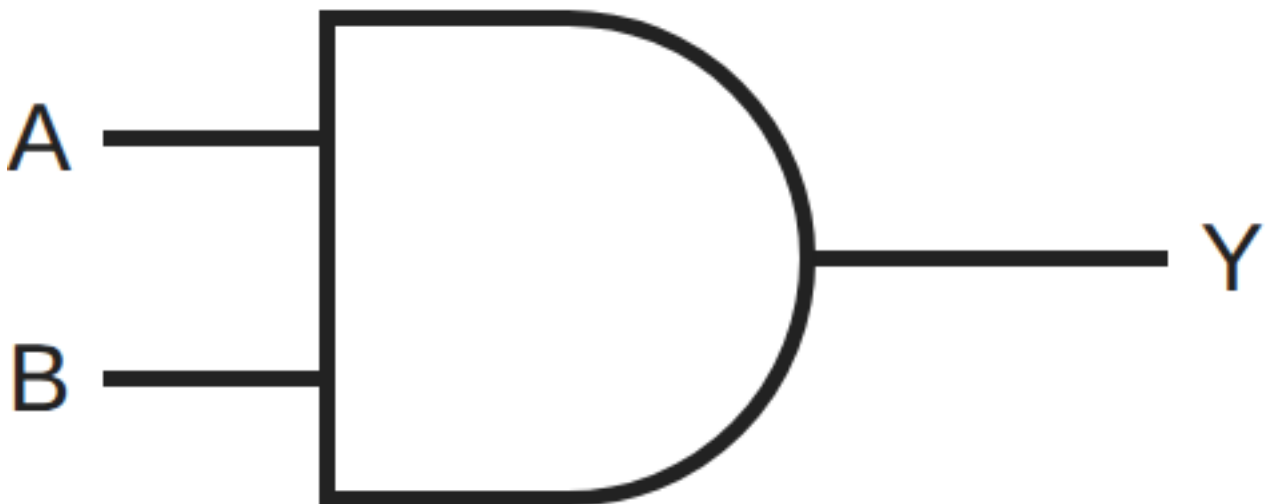
Símbolo (izquierda: ANSI/IEEE) · **Expresión:** $Y = A + B$



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

AND El resultado es 1 solo cuando ambas entradas son 1.

Símbolo (izquierda: ANSI/IEEE) · **Expresión:** $Y = A \cdot B$



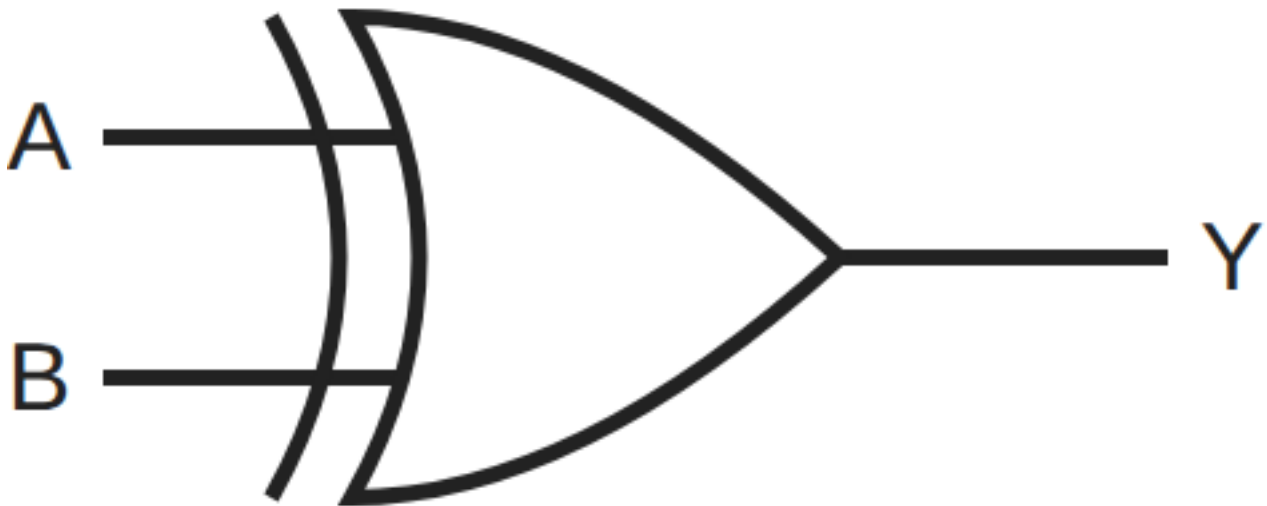
A	B	Y
0	0	0
0	1	0
1	0	0

A	B	Y
1	1	1

XOR (OR exclusiva) $A \text{ XOR } B = (\text{NOT}(A) \text{ AND } B) \text{ OR } (A \text{ AND NOT}(B))$

El resultado es 0 cuando ambas entradas son iguales y 1 cuando son diferentes.

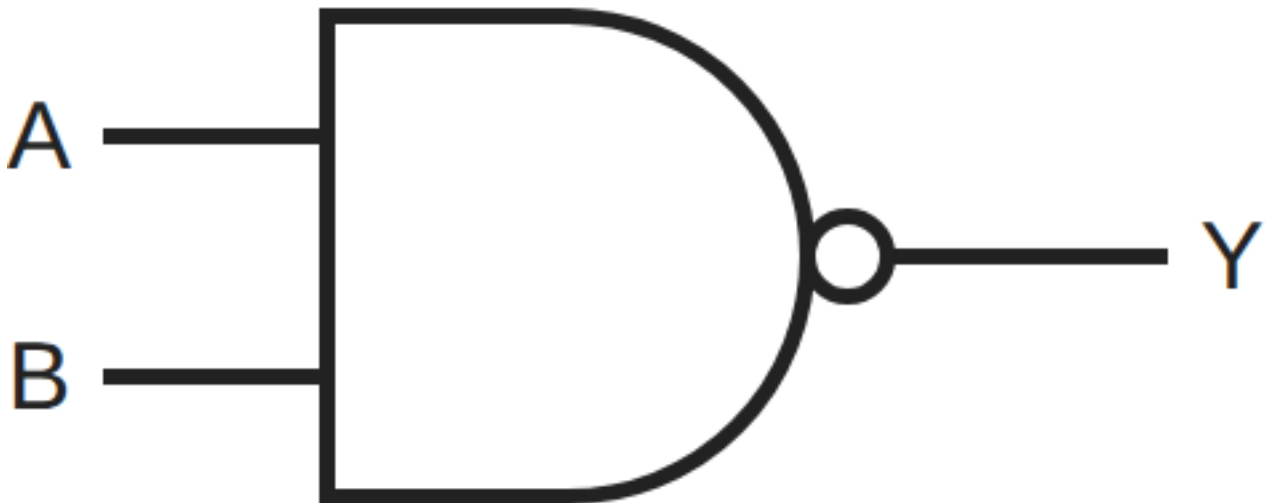
Símbolo (izquierda: ANSI/IEEE) · **Expresión:** $Y = A \oplus B$



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

NAND Negación de la AND: $A \text{ NAND } B = \text{NOT}(A \text{ AND } B)$. El resultado es 0 solo cuando ambas entradas son 1.

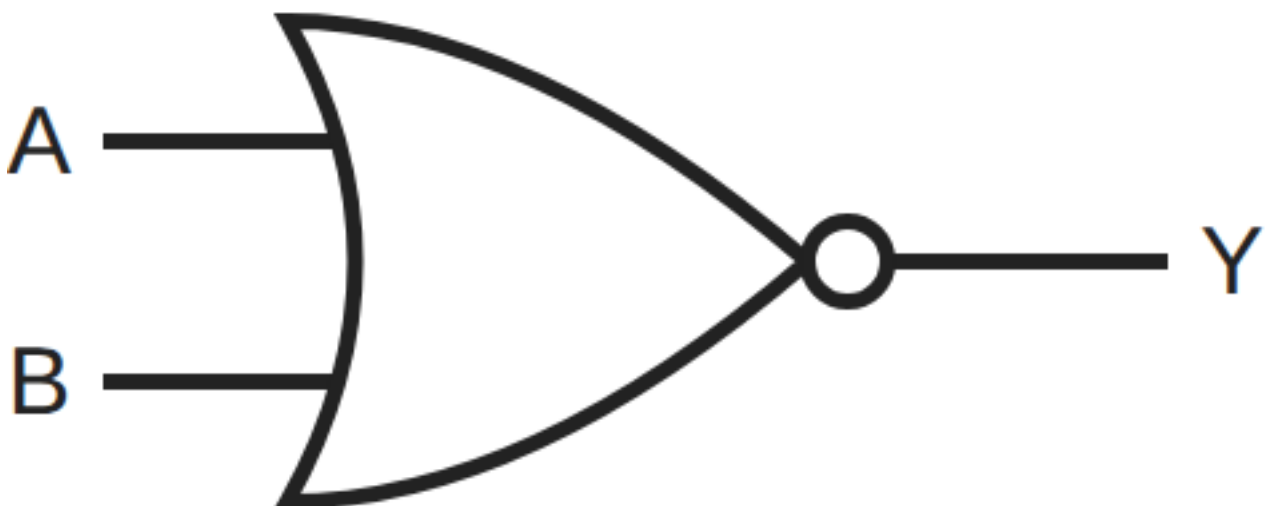
Símbolo (izquierda: ANSI/IEEE) · **Expresión:** $Y = \overline{A \cdot B}$



A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

NOR Negación de la OR: $A \text{ NOR } B = \text{NOT}(A \text{ OR } B)$. El resultado es 1 solo cuando ambas entradas son 0.

Símbolo (izquierda: ANSI/IEEE) · **Expresión:** $Y = \overline{A + B}$



A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

Ejemplo**Enunciado**

Para los siguientes pares de números binarios, realiza las operaciones AND, OR, NAND, NOR y XOR:

- Par 1: 1110 1100 y 0001 1101
- Par 2: 0110 1100 y 0010 1101

1.4.3 Complementos El ordenador utiliza los complementos tanto para **representar números negativos** como para **restar**, porque es más sencillo construir el circuito del complemento y la suma que el de la resta.

Para restar un número a otro, se suma el primero con el complemento del que se quiere restar; el resultado es idéntico al de la resta.

1.4.3.1 Complemento a 1 (C_1) Se obtiene cambiando los 0 por 1 y los 1 por 0; es decir, aplicando la operación lógica NOT al número. Ambos números deben tener la misma longitud en bits.

Ejemplo (N = 8 bits)

$$0000\ 1001_2 \rightarrow C_1 = 1111\ 0110$$

1.4.3.2 Complemento a 2 (C_2) Resuelve un problema del complemento a 1 al representar negativos: en C_1 existen **dos representaciones del 0** (0000 0000 y 1111 1111).

El complemento a 2 es el complemento a 1 más 1:

$$C_2 = C_1 + 1$$

Ejemplo (N = 8 bits)

$$0000\ 1001_2 \rightarrow C_2 = 1111\ 0110 + 1 = 1111\ 0111$$

1.4.3.3 Resta en complemento a 1 Se convierte el sustraendo a C_1 y se suman. Si sobra un bit de acarreo, **se suma al resultado** (*end-around carry*).

$$0000\ 1000 - 0000\ 0011\ (8 - 3)$$

$$\begin{array}{rcl}
 0000\ 1000 & & \\
 +\ 1111\ 1100 & <- C_1\ de\ 0000\ 0011 \\
 \hline
 1\ 0000\ 0100 & & \\
 \\
 0000\ 0100 & & \\
 +\ \quad\quad 1 & <-\ acarreo\ sobrante \\
 \hline
 0000\ 0101 & =\ 5
 \end{array}$$

1.4.3.4 Resta en complemento a 2 Se convierte el sustraendo a C_2 y se suman. Si sobra un bit de acarreo, **se descarta**.

0000\{\} 1000 - 0000\{\} 0011 (8 - 3)

$$C_2(0000\ 0011) = 1111\ 1100 + 1 = 1111\ 1101$$

0000 1000

+ 1111 1101

1 0000 0101 -> el 1 se descarta = 0000 0101 = 5

1.5 Detección de errores

Durante el almacenamiento o la transmisión pueden producirse errores: se almacena un 1 y al leerlo es un 0, o viceversa. Para detectarlos (e incluso corregirlos) se usan distintos métodos.

Todos los métodos de detección **añaden redundancia** al código: bits que no almacenan información, pero sirven para detectar errores.

1.5.1 Comprobación de paridad Consiste en contar cuántos “1” hay almacenados o recibidos y comprobar si esa cantidad es par o impar, según el tipo de paridad usado.

1.5.1.1 Paridad lineal Detecta si ha cambiado un bit. Para generar el bit de paridad se utiliza la operación XOR. Se añade **un bit** según el criterio adoptado:

- **Par:** el número de bits a 1 (incluido el de paridad) debe ser par.

$$p = b_1 \text{ XOR } b_2 \text{ XOR } \dots \text{ XOR } b_n$$
- **Impar:** el número de bits a 1 (incluido el de paridad) debe ser impar.

$$p = \text{NOT}(b_1 \text{ XOR } b_2 \text{ XOR } \dots \text{ XOR } b_n)$$

Es un método sencillo, pero solo detecta un número impar de errores (un error simple) y además **no indica qué bit es el erróneo**.

1.5.1.2 Paridad bidimensional Mejora el sistema anterior: se calcula el bit de paridad tanto para cada **fila** como para cada **columna**. Puede detectar más errores y localizar dónde están.

Ejemplo

Para almacenar los siguientes bits con paridad bidimensional **par**, ¿qué tendríamos que añadir?

0010011

0101111

0001000

1.5.2 Códigos cíclicos (CRC) Los **CRC** (*Códigos de Redundancia Cíclica*) incorporan como bits de control el resultado de un cálculo basado en la operación módulo (resto de la división entre dos números/polinomios, usando XOR sin acarrees).

La eficiencia depende del **polinomio generador** elegido. Destacan: **CRC-12**, **CRC-16**, **CRC-32**, etc.

Actualización 2026

El CRC sigue muy vigente: **CRC-32** se usa en Ethernet (IEEE 802.3), Wi-Fi (IEEE 802.11), SATA, PCI Express, ZIP, gzip y PNG, y su variante **CRC-32C** en iSCSI, NVMe o los sistemas de archivos ext4 y Btrfs, para detectar corrupción de datos (fuente: [Cyclic redundancy check - Wikipedia](#)).

Al enviar o almacenar los datos se añaden bits de control (tantos como el grado del polinomio generador) que no aportan información. Frente a la paridad, **detectan más errores**, pero el código es **más redundante**.

Ejemplo CRC-3

- Datos $D(X)$: 101110

- Polinomio generador $G(X)$: 1001 (grado 3 → se añaden 3 bits)

Generación: se añaden 3 ceros a los datos y se divide (XOR) entre $G(X)$:

```
101110000 | 1001
1001       | 101011 (cociente)
```

```
----
0101
0000
```

```
----
1010
1001
```

```
----
0110
0000
```

```
----
1100
1001
```

```
----
1010
1001
```

```
----
011  <- resto R(X)
```

Se transmite: **101110011** (datos + resto).

Comprobación: el receptor divide 101110011 entre 1001. El resto es 000 → **sin errores**.

2. Codificación de la información

Codificar es transformar la información de un tipo de representación a otro.

Existen dos tipos de códigos:

- **Códigos de longitud fija:** todos los datos ocupan el mismo espacio, es decir, se usa siempre el mismo número de símbolos para representar cualquier dato. Ejemplo: ASCII o BCD.
- **Códigos de longitud variable:** cada dato puede ocupar un espacio distinto, porque el número de símbolos usado es variable. Ejemplo: código Morse (A: . -, S: ...).

Actualización 2026

El ASCII original es de **7 bits** (128 caracteres) y solo cubre el inglés. Hoy el estándar universal es **Unicode** (versión **18.0**, septiembre de 2026, con **172 808** caracteres), y su codificación más usada es **UTF-8**, que es de **longitud variable** (1 a 4 bytes por carácter) y compatible con ASCII: los 128 primeros caracteres ocupan 1 byte igual que en ASCII. UTF-8 la usa el **99,1 %** de los sitios web (fuentes: [Unicode 18.0.0](#), [W3Techs - UTF-8](#)).

En informática, codificar es convertir un carácter o número de nuestro lenguaje en un conjunto de ceros y unos que el ordenador sí comprende. Según se quiera almacenar un número o un carácter, se usan distintos métodos y códigos.

2.1 Almacenamiento de la información

Un dígito binario es un **bit** (*binary digit*): la unidad de información más pequeña. Con un bit solo podemos almacenar un 0 o un 1, así que para almacenar más información se usan unidades mayores:

- *Nibble* o cuarteto: 4 bits.
- *Byte* u octeto: 8 bits.

Desde el inicio de la informática se cometió el “error” de aproximar $1 \text{ KB} = 2^{10} \text{ B} = 1024 \text{ B}$. Dada su rápida adopción, el Sistema Internacional propuso como solución que **1 kilobyte = 1000 bytes** y que, si la comunidad informática quería sus propios múltiplos binarios, usara $2^{10} \text{ B} = 1 \text{ KiB}$ (*kibibyte*), definidos por la IEC.

Múltiplos del byte

Múltiplo SI (símbolo)	Valor	Múltiplo binario IEC (símbolo)	Valor
kilobyte (kB)	10^3	kibibyte (KiB)	2^{10}
megabyte (MB)	10^6	mebibyte (MiB)	2^{20}
gigabyte (GB)	10^9	gibibyte (GiB)	2^{30}
terabyte (TB)	10^{12}	tebibyte (TiB)	2^{40}
petabyte (PB)	10^{15}	pebibyte (PiB)	2^{50}
exabyte (EB)	10^{18}	exbibyte (EiB)	2^{60}
zettabyte (ZB)	10^{21}	zebibyte (ZiB)	2^{70}
yottabyte (YB)	10^{24}	yobibyte (YiB)	2^{80}

Actualización 2026

- En 2022 la CGPM añadió los prefijos SI **ronna** (R, 10^{27}) y **quetta** (Q, 10^{30}); no hay aún prefijos binarios IEC equivalentes (fuente: [BIPM - Resolución 3 de la CGPM 2022](#)).
- Ojo con el uso práctico: la capacidad de los discos se anuncia con prefijos SI ($1 \text{ TB} = 10^{12} \text{ B}$), pero si el sistema operativo cuenta en múltiplos de 1024 mostrará $10^{12}/2^{30} \approx 931 \text{ GiB}$, a menudo rotulados incorrectamente como “GB”. Los prefijos kibi, mebi, gibi... están definidos por la IEC (fuente: [NIST - Prefixes for binary multiples](#)).

2.2 Codificación numérica

Para codificar internamente los números se emplean distintas formas según sea entero o tenga decimales, y según su valor.

2.2.1 Números enteros

BCD (Decimal Codificado en Binario) Asigna **4 bits** a cada dígito decimal. Si el número tiene varios dígitos, se usan tantos grupos de 4 bits como dígitos tenga.

Decimal	BCD	Decimal	BCD
0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

Ejemplo

$568_{10} \rightarrow 0101\ 0110\ 1000_{BCD}$

Decimal desempaquetado Usa **8 bits (1 byte)** por dígito: el nibble izquierdo (zona) vale 1111 y el derecho contiene el dígito en BCD. El signo se codifica en el nibble izquierdo del byte de más a la derecha (el nibble más significativo del byte menos significativo): 1100 = +, 1101 = -.

Ejemplo

- $+3457_{10} \rightarrow 1111\ 0011\ 1111\ 0100\ 1111\ 0101\ 1100\ 0111_2$ (3, 4, 5, +7)
 - $-3457_{10} \rightarrow 1111\ 0011\ 1111\ 0100\ 1111\ 0101\ 1101\ 0111_2$ (3, 4, 5, -7)

Decimal empaquetado Usa **4 bits (1 nibble)** por dígito. El signo se representa con un nibble adicional a la derecha: 1100 = +, 1101 = -.

Ejemplo

- $+3457_{10} \rightarrow 0011\ 0100\ 0101\ 0111\ 1100_2$ (3, 4, 5, 7, +)
 - $-3457_{10} \rightarrow 0011\ 0100\ 0101\ 0111\ 1101_2$ (3, 4, 5, 7, -)

Binario puro con signo (signo-magnitud) Los ordenadores actuales suelen usar 4 bytes (32 bits) para representar un entero con signo. El número se almacena en binario puro más un **bit de signo**:

- 31 bits para el valor.
- El bit de la izquierda (el más significativo) para el signo: **0 = positivo, 1 = negativo**.

Por tanto, con 4 bytes:

- Mayor entero: $2^{31} - 1 = 2147483647$
- Menor entero: $-(2^{31} - 1) = -2147483647$

Dos ceros

En signo-magnitud existen +0 y -0. Según el sistema, los negativos también pueden almacenarse en complemento a 1 o a 2; en complemento a 2 (el más habitual) el rango con 32 bits va de -2^{31} a $2^{31} - 1$.

Ejemplo: representa en binario puro con signo (4 bytes)

$$\begin{aligned} - +15678_{10} &\rightarrow \mathbf{0}0000000 \ 00000000 \ 00111101 \ 00111110_2 \\ - -25896457_{10} &\rightarrow \mathbf{1}0000001 \ 10001011 \ 00100110 \ 00001001_2 \end{aligned}$$

2.2.2 Números con decimales

Los números con parte fraccionaria, o los enteros demasiado grandes (o muy pequeños) para la representación de enteros, se representan en **coma flotante**:

$$N = (-1)^{\text{signo}} \cdot \text{mantisa} \cdot \text{base}^{\text{exponente}}$$

- **Signo:** 0 si es positivo, 1 si es negativo.
- **Mantisa:** valor que, multiplicado por la base elevada al exponente, da el número.
- **Base:** 2 en binario, 10 en decimal.
- **Exponente:** número entero (positivo o negativo).

Según los bits usados, la precisión es mayor o menor:

- **Simple precisión:** 32 bits (4 bytes).
- **Doble precisión:** 64 bits (8 bytes).

Notación científica normalizada En decimal, cualquier número real puede escribirse en **notación científica normalizada**: se multiplica por una potencia de 10 (de exponente positivo o negativo) de forma que a la izquierda de la coma quede **un único dígito distinto de 0** y el resto a la derecha. P. ej. $48,125 = 4,8125 \cdot 10^1$.

En binario hay varios métodos para almacenar números en coma flotante; el más utilizado es el estándar **IEEE 754**.

IEEE 754

Formato	Tamaño	Signo	Exponente (sesgado)	Mantisa	Tipo
Simple precisión	4 bytes	1 bit	8 bits (127)	23 bits	float
Doble precisión	8 bytes	1 bit	11 bits (1023)	52 bits	double

- **Signo:** 0 positivo, 1 negativo.
- **Exponente (sesgado):** se almacena $e + (2^{n-1} - 1)$, donde e es el exponente de 2 resultante de desplazar la coma y n el número de bits del exponente.
 - Simple precisión: $n = 8 \Rightarrow 2^7 - 1 = 127$
 - Doble precisión: $n = 11 \Rightarrow 2^{10} - 1 = 1023$
- **Mantisa (bit implícito):** se desplaza la coma hasta dejarla justo a la derecha del primer 1 ($1,xxxx$). Ese 1 **no se almacena** (se sabe que siempre es 1), con lo que se gana un bit de precisión. Se guardan solo los bits tras la coma.

La secuencia normalizada en IEEE 754 para la transmisión de información tiene la siguiente secuencia y con las mismas características:

Formato	Signo S (bits)	Exponente E (bits)	Mantisa M (bits)
Simple precisión	bit 31 (1)	bits 30-23 (8)	bits 22-0 (23)
Doble precisión	bit 63 (1)	bits 62-52 (11)	bits 51-0 (52)

Dato a trasmitir = SEEEEEEE EMMMMMMM MMMMMMMM MMMMMMMM

- *S*: Signo
- *E*: Exponente
- *M*: Mantisa

Ejemplo 1: 48{,}125_{10} en simple precisión

1. Binario: $48,125_{10} = 110000,001_2$
 2. Normalizar: $1,10000001 \cdot 2^5$
 3. Signo: 0
 4. Exponente: $5 + 127 = 132 = 10000100_2$
 5. Mantisa (sin el 1 implícito, rellenada a 23 bits): 10000001000000000000000
- Resultado: 0 10000100 10000001000000000000000
- Dato a transmitir: 01000010 01000000 10000000 00000000 (42408000_{16})

Ejemplo 2: -0{,}00000111_2 en simple precisión

1. Normalizar: $-1,11 \cdot 2^{-6}$
 2. Signo: 1
 3. Exponente: $-6 + 127 = 121 = 01111001_2$
 4. Mantisa: 11000000000000000000000
- Resultado: 1 01111001 11000000000000000000000
- Dato a transmitir: 10111100 11100000 00000000 00000000 ($BCE00000_{16}$, es decir, $-0,02734375_{10}$)

Actualización 2026

La versión vigente del estándar es **IEEE 754-2019** (sustituye a la de 2008; publicada también como ISO/IEC 60559:2020). Define formatos de intercambio y aritmética tanto **binarios** (simple = *binary32*, doble = *binary64*) como **decimales** (fuente: [IEEE SA - IEEE 754-2019](#)).